

Sanitized Technical Memo: Reproducible Cohort + Lookback Pipeline in a Restricted Data Environment

Author: Jiayu Feng | Date: 2026-02-20

Purpose

This memo summarizes an end-to-end workflow used to (1) construct an analysis cohort and (2) compute chronic condition indicators across multiple lookback windows using published clinical/administrative specifications. The underlying work was performed inside a restricted data enclave, so all dataset identifiers, code, and outputs are intentionally omitted or generalized.

Operating constraints (restricted environment)

- No raw record-level export; only approved aggregated outputs can leave the enclave.
- Strict privacy, compliance, and auditing requirements; all logic must be traceable and rerunnable.
- Reproducibility is a first-class requirement (parameterized runs, versioned specs, and documented QA).

High-level cohort definition (sanitized)

- Start from a large administrative eligibility/demographics dataset and define an adult working-age cohort using an end-of-year index date.
- Apply continuous enrollment criteria (e.g., minimum consecutive enrollment months) to ensure sufficient observation for lookbacks.
- Apply a policy-relevant eligibility filter based on standard eligibility group categories (exact codes omitted).
- Deduplicate to beneficiary level and retain key stratifiers (e.g., state) for aggregated reporting.

Lookback windows and condition indicators

Condition indicators are computed using published algorithmic definitions (e.g., chronic condition groupers) across multiple lookback windows (e.g., multi-year, annual, semi-annual, and quarterly windows). Window definitions are parameterized so the same code can rerun with different index dates and lookback lengths.

Pipeline architecture (high level)

- Extract & standardize: assemble claims/encounter sources required by the published condition algorithms (exact file names omitted).
- Transform: implement specification-to-code modules that generate intermediate tables used by the algorithms.
- Compute: produce condition flags by window, ensuring consistent logic and version control for code lists and parameters.
- Aggregate & report: generate overall and geography-stratified summaries using privacy-preserving aggregation and suppression-aware rules when required.

Quality assurance and auditability

- Spec traceability: maintain a mapping from each algorithm/spec section to the corresponding code module and parameters.
- Cohort integrity checks: validate enrollment rules, deduplication, and attrition accounting at each stage.
- Window-consistency checks: verify nestedness relationships (shorter windows are subsets of longer windows) and investigate deviations.
- Reproducibility controls: parameter files, deterministic reruns, and decision logs for anomalies or spec ambiguities.

Why this is relevant (transferable skills)

This work emphasizes rigorous measurement under constraints: translating formal specifications into scalable, testable code; building evaluation-ready datasets; and producing privacy-aware, audit-friendly outputs. These skills transfer directly to high-stakes evaluation and monitoring workflows, where validity, reproducibility, and governance matter as much as model performance.

Disclosure: This memo is intentionally sanitized and does not include employer/PI information, restricted dataset identifiers, code, unpublished results, or cell-level outputs.